

Chapitre N°4 : Système de numérisation

1. Conversation et codage de l'information

Les informations traitées par les ordinateurs sont de différentes natures :

- Nombres, texte,
- Images, sons, vidéo,
- Programmes, ...
- Dans un ordinateur, elles sont toujours représentées sous forme binaire (BIT : Binary digIT)
- Une suite de 0 et de 1.

a. Codage de l'information

Le codage de l'information permet d'établir une correspondance qui permet sans ambiguïté de passer d'une représentation (dite externe) d'une information à une autre représentation (dite interne : sous forme binaire) de la même information, suivant un ensemble de règles précises.

Exemple :

- Le nombre 35 : 35 est la représentation externe du nombre trente-cinq.
- La représentation interne de 35 sera une suite de 0 et 1 (100011).

En informatique, Le codage de l'information s'effectue principalement en trois étapes :

1. L'information sera exprimée par une suite de nombres (**Numérisation**).
2. Chaque nombre est codé sous forme **binaire** (suite de 0 et 1).
3. Chaque élément binaire est représenté par un état **physique**.

Le codage de l'élément binaire par un état physique peut se faire par :

- Charge électrique (RAM : Condensateur-transistor) : Chargé (bit 1) ou non chargé (bit 0)
- Magnétisation (Disque dur, disquette) : polarisation Nord (bit 1) ou Sud (bit 0)
- Alvéoles (CDROM) : réflexion (bit 1) ou pas de réflexion (bit 0)
- Fréquences (Modem) : dans un signal sinusoïdal.

b. Système de numération

Système de numération décrit la façon avec laquelle les nombres sont représentés.

Un système de numération est défini par :

- Un alphabet $\mathcal{A}$: ensemble de symboles ou chiffres,
- Des règles d'écritures des nombres : juxtaposition de symboles.

i. Exemples de Système de numération

- Numération Romaine : I, II, III, IV, ..., X, ..., CCLXXI $\rightarrow$ 271
 - Numération babylonienne :  , ..., 
 - Numération décimale : $\mathcal{A} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 - Le nombre 10 est la base de cette numération
 - C'est un système positionnel. Chaque position possède un poids.
- ii. Système de numération positionnel pondéré à base b

Un système de numérotation positionnel pondéré à base b est défini sur un alphabet de b chiffres:

$$\mathcal{A} = \{c_0, c_1, \dots, c_{b-1}\} \text{ avec } 0 \leq c_i < b$$

- Soit $N = a_{n-1} a_{n-2} \dots a_1 a_0 (b)$: représentation en base b sur n chiffres. $\circ a_i$: est un chiffre de l'alphabet de poids i (position i). $\circ a_0$: chiffre de poids 0 appelé le chiffre de poids faible. $\circ a_{n-1}$: chiffre de poids $n-1$ appelé le chiffre de poids fort.
- La valeur de N en base 10 est donnée par :

$$N = a_{n-1}.b_{n-1} + a_{n-2}.b_{n-2} + \dots + a_0.b_0 \quad (10) = \sum_{nn-1ii=0} aa_{ii} . bb_{ii}$$

iii. Bases de numération

- **Système binaire** : système à base 2 ($b=2$) utilise deux chiffres : $\{0,1\}$ ○ C'est avec ce système que fonctionnent les ordinateurs
- **Système Octal** : système à base 8 ($b=8$) utilise huit chiffres : $\{0,1,2,3,4,5,6,7\}$
 - Utilisé il y a un certain temps en Informatique. ○ Elle permet de coder **3 bits** par un seul symbole.
- **Système Hexadécimal** : système à base 16 ($b=16$) utilise 16 chiffres : $\{0,1,2,3,4,5,6,7,8,9, A=10_{(10)}, B=11_{(10)}, C=12_{(10)}, D=13_{(10)}, E=14_{(10)}, F=15_{(10)}\}$
 - Cette base est très utilisée dans le monde de la micro-informatique. ○ Elle permet de coder **4 bits** par un seul symbole.

iv. Transcodage (conversion de base)

Le transcodage ou conversion de base est l'opération qui permet de passer de la représentation d'un nombre exprimé dans une base à la représentation du même nombre mais exprimé dans une autre base.

Par la suite, on verra les conversions suivantes :

- Décimal vers Binaire, Octal et Hexadécimal.
- Binaire vers Décimal, Octale et Hexadécimal.

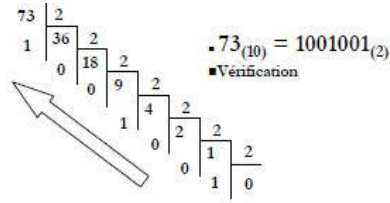
1. De la base 10 vers une base b

La règle à suivre est les divisions successives :

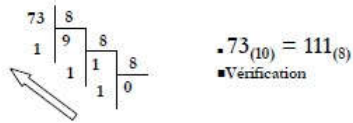
- On divise le nombre par la base b .
- Puis le quotient par la base b .
- Ainsi de suite jusqu'à l'obtention d'un quotient nul.
- La suite des restes correspond aux symboles de la base visée.
- On obtient en premier le chiffre de poids faible et en dernier le chiffre de poids fort.

► **Exemple** : Soit N le nombre d'étudiants d'une classe représenté en base décimale par : N
 $= 73_{(10)}$

Représentation en Binaire ?



Représentation en Octal ?



Représentation en Hexadécimal ?



2. De la base 2 vers une base b

- **Solution 1** : convertir le nombre en base binaire vers la base décimale puis convertir ce nombre en base 10 vers la base b.

- Exemple :

$$10010_{(2)} = ?_{(8)}$$

$$10010_{(2)} = 2^4 + 2^1 = 16 + 2 = 18_{(10)} = 2 \times 8_1 + 2 \times 8_0 = 22_{(8)}$$

- **Solution 2** :

- Binaire vers décimal : par définition ($\sum_{i=0}^{n-1} a_i 2^i$)

► **Exemple** : Soit N = $1010011101_{(2)} = ?_{(10)}$

$$N = 1 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 512 + 0 + 128 + 0 + 0 + 16 + 8 + 4 + 0 + 1$$

$$= 669_{(10)}$$

$$\Rightarrow 1010011101_{(2)} = 669_{(10)}$$

- Binaire vers octal : regroupement des bits en des sous-ensembles de trois bits puis remplacer chaque groupe par le symbole correspondant dans la base 8. (Voir tableau ci-contre)

► **Exemple** : Soit N = $1010011101_{(2)} = ?_{(8)}$

$$N = 001\ 010\ 011\ 101_{(2)}$$

$$= 1\ 2\ 3\ 5_{(8)}$$

$$\Rightarrow 1010011101_{(2)} = 1235_{(8)}$$

- Binaire vers Hexadécimal : regroupement des bits en des sous-ensembles de quatre bits puis remplacer chaque groupe par le symbole correspondant dans la base 16. (Voir tableau ci-contre).

► **Exemple** : Soit N = $1010011101_{(2)} = ?_{(16)}$

$$N = 0010\ 1001\ 1101_{(2)}$$

$$= 2\ 9\ D_{(16)}$$

$$\Rightarrow 1010011101_{(2)} = 29D_{(16)}$$

Symbole Octale	Suite binaire
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Symbole Hexadécimal	Suite binaire
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

c. Codage des nombres

i. Codage des entiers naturels

1. Utilisation du code binaire pur :

- L'entier naturel (positif ou nul) est représenté en base 2, ○ Les bits sont rangés selon leur poids, on complète à gauche par des 0.

► Exemple : sur un octet, $10_{(10)}$ se code en binaire pur :

$$00001010_{(2)}$$

2. Etendu du codage binaire pur :

- Sur n bits on peut coder des nombres de 0 à $2^n - 1$ ○ Sur 1 octet (8 bits) : codage des nombres de 0 à $2^8 - 1 = 255$ ○ Sur 2 octets (16 bits) : codage des nombres de 0 à $2^{16} - 1 = 65535$ ○ Sur 4 octets (32 bits) : codage des nombres de 0 à $2^{32} - 1 = 4\ 294\ 967\ 295$

3. Arithmétique en base 2

- Les opérations sur les entiers s'appuient sur des tables d'addition et de multiplication

Addition		
0	0	0
0	1	1
1	0	1
1	1	0

Retenue

► Exemple (Addition)

Addition binaire (8 bits)

```

  1 0 0 1 0 1 1 0
+ 0 1 0 1 0 1 0 1
-----
  1 1 1 0 1 0 1 1

```

► Exemple (Multiplication)

Multiplication binaire

```

  1 0 1 1 (4 bits)
× 1 0 1 0 (4 bits)
-----

```

```

      0 0 0 0
    1 0 1 1 .
    0 0 0 0 .
  1 0 1 1 .
-----
  0 1 1 0 1 1 0

```

Sur 4 bits le résultat est faux
 Sur 7 bits le résultat est juste
 Sur 8 bits on complète à gauche par un 0

ii. Codage des entiers relatifs

Il existe au moins trois façons pour coder un entier relatif (signé) :

- Code binaire signé (par signe et valeur absolue).
- Code complément à 1.
- Code complément à 2 (utilisé sur ordinateur).

1. Binaire signé

► Le bit le plus significatif est utilisé pour représenter le signe du nombre :

- Si le bit le plus fort = 1 alors nombre négatif
- Si le bit le plus fort = 0 alors nombre positif

► Les autres bits codent la valeur absolue du nombre.

► Exemple : Sur 8 bits, codage des nombres -24 et -128 en (bs)

-24 est codé en binaire signé par : 1 0 0 1 1 0 0 0_(bs) -128

hors limite nécessite 9 bits au minimum.

► Etendu de codage :

- Avec n bits, on code tous les nombres entre $-(2^{n-1}-1)$ et $(2^{n-1}-1)$.
- Avec 4 bits : -7 et +7.

► Limitations du binaire signé :

- Deux représentations du zéro : +0 et -0
- Sur 4 bits : +0 = 0000_(bs), -0 = 1000_(bs)
- Multiplication et l'addition sont moins évidentes.

2. Code complément à 1

Aussi appelé Complément Logique (CL) ou Complément Restreint (CR) :

- Les nombres positifs sont codés de la même façon qu'en binaire pur.
- Un nombre négatif est codé en **inversant** chaque bit de la représentation de sa valeur absolue.

► Le bit le plus significatif est utilisé pour représenter le signe du nombre :

- Si le bit le plus fort = 1 alors nombre négatif.
- Si le bit le plus fort = 0 alors nombre positif. ► Exemple : -24 en complément à 1 sur 8 bits

$|-24|$ en binaire pur $\Rightarrow 00011000_{(2)}$, puis on inverse les bits $\Rightarrow 11100111_{(c\bar{a}1)}$

► Limitations du complément à 1 :

- Deux codages différents pour 0 (+0 et -0)
- Sur 8 bits : +0 = 00000000_(c\bar{a}1) et -0 = 11111111_(c\bar{a}1)
- Multiplication et l'addition sont moins évidentes.

3. Code complément a 2

Aussi appelé **Complément Vrai** (CV) :

- Les nombres positifs sont codés de la même manière qu'en binaire pur.
- Un nombre négatif est codé en ajoutant la valeur 1 à son complément à 1.

► Le bit le plus significatif est utilisé pour représenter le signe du nombre. ► Exemple : -24 en complément à 2 sur 8 bits

24 est codé par 00011000₍₂₎

-24 $\Rightarrow 11100111_{(c\bar{a}1)}$, puis on lui additionne 1 donc -24 est codé par 11101000_(c\bar{a}2)

► Un seul codage pour 0. Par exemple sur 8 bits :

- +0 est codé par 00000000_(c\bar{a}2)
- -0 est codé par 11111111_(c\bar{a}1) $\Rightarrow$ donc -0 sera représenté par 00000000_(c\bar{a}2)

► Etendu de codage :

- Avec n bits, on peut coder de $-(2^{n-1})$ à $(2^{n-1}-1)$
- Sur 1 octet (8 bits), codage des nombres de -128 à 127
 - +0 = 00000000 -0 = 00000000
 - +1 = 00000001 -1 = 11111111
 - +127 = 01111111 -128 = 10000000

iii. Codage des nombres réels

Les formats de représentations des nombres réels sont :

- **Format virgule fixe** : (utilisé par les premières machines) ○ Possède une partie « entière » et une partie « décimale » séparés par une virgule.
 - La position de la virgule est fixe d'où le nom.
 - Exemple : $54,25_{(10)}$; $10,001_{(2)}$; $A1,F0B_{(16)}$
 - **Format virgule flottante** : (utilisé actuellement sur machine) ○ Défini par : $\pm m \cdot b^e$
 - un signe + ou -
 - une mantisse **m** (en virgule fixe)
 - un exposant **e** (un entier relative)
 - une base **b** (2,8,10,16,...)
- Exemple : $0,5425 \cdot 10^2_{(10)}$; $10,1 \cdot 2^{-1}_{(2)}$; $A0,B4 \cdot 16^{-2}_{(16)}$

1. Codage en Virgule Fixe

Etant donné une base b, un nombre X est représenté, en format virgule fixe, par :

- $X = a_{n-1} a_{n-2} \dots a_1 a_0 , a_{-1} a_{-2} \dots a_{-p}^{(b)}$
- a_{n-1} est le chiffre de poids fort (MSB⁴).
- a_{-p} est le chiffre de poids faible (LSB⁵).
- n est le nombre de chiffre avant la virgule.
- p est le nombre de chiffre après la virgule.
- La valeur de X en base 10 est : $X = \sum_{i=0}^{n-1} a_i b^i + \sum_{i=1}^p a_{-i} b^{-i}$

► Exemple : $101,01_{(2)} = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 5,25_{(10)}$

✓ Changement de base 10 $\rightarrow$ 2

- Le passage de la base 10 à la base 2 est défini par :
 - Une partie entière codée sur p bits (division successive par 2)
 - Une partie décimale codée sur q bits en multipliant par 2 successivement jusqu'à ce que la partie décimale soit nulle ou le nombre de bits q est atteint.

► Exemple : $4,25_{(10)} = ?_{(2)}$ format virgule fixe

$$\begin{aligned} 4_{(10)} &= 100_{(2)} \\ 0,25 \times 2 &= 0,5 \rightarrow 0 \text{ } 0,5 \\ \times 2 &= 1,0 \rightarrow 1 \\ \Rightarrow \text{donc } 4,25_{(10)} &= 100,01_{(2)} \end{aligned}$$

2. Codage en Virgule Flottante

Un nombre réel est représenté en virgule flottante sous la forme : $\pm M \times 2^E$, où M est la mantisse (virgule fixe) et E l'exposant (signé). Coder en base 2 au format virgule flottante, revient à coder le signe, la mantisse et l'exposant.

► Exemple : Codage en base 2, format virgule flottante, du nombre 3,25

$$\begin{aligned} 3,25_{(10)} &= 11,01_{(2)} \text{ (en virgule fixe)} \\ &= 1,101 \times 2^1_{(2)} \rightarrow \text{on décale la virgule par 1 bit à gauche.} \\ &= 110,1 \times 2^{-1}_{(2)} \rightarrow \text{on décale la virgule par 1 bit à droite.} \end{aligned}$$

Problème : il y a différentes manières de représenter E et M.

⁴ Most significant bit

⁵ Less significant bit

⇒ **Normalisation** : afin d'avoir la même représentation, tout nombre doit d'abord être normalisé sous la forme : $\pm 1, M \times 2^{Eb}$

- Le signe est codé sur 1 bit ayant le poids fort (**S**):
 - Le signe - : bit 1
 - Le signe + : bit 0
- Exposant biaisé (**Eb**) :
 - Placé avant la mantisse pour simplifier la comparaison.
 - Codé sur p bits et biaisé pour être positif (ajout de $2^{p-1}-1$).
- Mantisse normalisée (**M**) :
 - Normalisée : la virgule est placée après le bit à 1 ayant le poids fort.
 - M est codé sur q bits ○ Exemple : $11,01 \Rightarrow 1,101 \times 2^1$ donc M = 101

S	Eb	M
1bit	p bits	q bits

► Standard IEEE 754 (1985)

C'est la norme la plus employée actuellement pour le calcul des nombres à virgule flottante avec les CPU. Elle définit les nombres réels sur les formats :

- **Simple précision**, codé sur 32 bits :
 - 1 bit de signe (**S**). ○ 8 bits pour l'exposant biaisé (**Eb**).
 - 23 bits pour la mantisse (**M**).

S	Eb	M
1bit	8 bits	23 bits

- **Double précision**, codé sur 64 bits :
 - 1 bit de signe (**S**). ○ 11 bits pour l'exposant biaisé (**Eb**).
 - 52 bits pour la mantisse (**M**).

S	Eb	M
1bit	11 bits	52 bits

► Conversion décimale - IEEE754 (Codage d'un réel)

$35,5_{(10)} = ?_{(2)}$ (IEEE 754 simple précision)

Nombre positif, $\Rightarrow$ donc S = 0

$35,5_{(10)} = 100011,1_{(2)}$ (virgule fixe)

$= 1,000111 \times 2^5_{(2)}$ (virgule flottante)

Exposant = Eb-127 = 5, donc Eb = 132 1,M

$= 1,000111$ donc M = 00011100...

0 10000100 000111000000000000000000 (IEEE 754 SP)

S	Eb	M
---	----	---

► Conversion IEEE754 – Décimale (Evaluation d'un réel)

0 10000001 111000000000000000000000 (IEEE 754 SP)

S	Eb	M
---	----	---

S = 0, donc nombre positif

Eb = 129, donc exposant = Eb-127 = 2

1,M = 1,111

$+ 1,111 \times 2^2_{(2)} = 111,1_{(2)} = 7,5_{(10)}$

► Caractéristiques des nombres flottants au standard IEEE

	Simple précision	Double précision
Bit de signe	1	1
Bits d'exposant	8	11
Bits de mantisse	23	52
Nombre total de bits	32	64
Codage de l'exposant	Excédant 127	Excédant 1023
Variation de l'exposant	-126 à +127	-1022 à +1023
Plus petit nombre normalisé	2^{-126}	2^{-1022}
Plus grand nombre normalisé	Environ 2^{+128}	Environ 2^{+1024}

d. Codage des caractères

Les caractères peuvent être soit Alphabétique (A-Z, a-z), soit numérique (0, ..., 9), des signes de ponctuation, ou des caractères spéciaux (&, \$, %, ...). Le codage des caractères revient à créer une table de correspondance entre les caractères et des nombres.

► Les Standards :

• Code (ou Table) **ASCII** (American Standard Code for Information Interchange) :

- 7 bits pour représenter 128 caractères (0 à 127)
- 48 à 57 : chiffres dans l'ordre (0, 1, ..., 9)
- 65 à 90 : les alphabets majuscules (A, ..., Z)
- 97 à 122 : les alphabets minuscule (a, ..., z)

MSB	8	9	A	B	C	D	E	F
LSB	1000	1001	1010	1011	1100	1101	1110	1111
0	0000	0	E	a	1	2	3	4
1	0001	1	2	3	4	5	6	7
2	0010	2	3	4	5	6	7	8
3	0011	3	4	5	6	7	8	9
4	0100	4	5	6	7	8	9	A
5	0101	5	6	7	8	9	A	B
6	0110	6	7	8	9	A	B	C
7	0111	7	8	9	A	B	C	D
8	1000	8	9	A	B	C	D	E
9	1001	9	A	B	C	D	E	F
A	1010	A	B	C	D	E	F	G
B	1011	B	C	D	E	F	G	H
C	1100	C	D	E	F	G	H	I
D	1101	D	E	F	G	H	I	J
E	1110	E	F	G	H	I	J	K
F	1111	F	G	H	I	J	K	L

	FE7	FE8	FE9	FEA	FEB	FDF
0	صلى	ز	ح	س	ق	ط
1	ط	ق	ح	س	ز	ص
2	ط	ق	ح	س	ز	ص
3	ط	ق	ح	س	ز	ص
4	ط	ق	ح	س	ز	ص

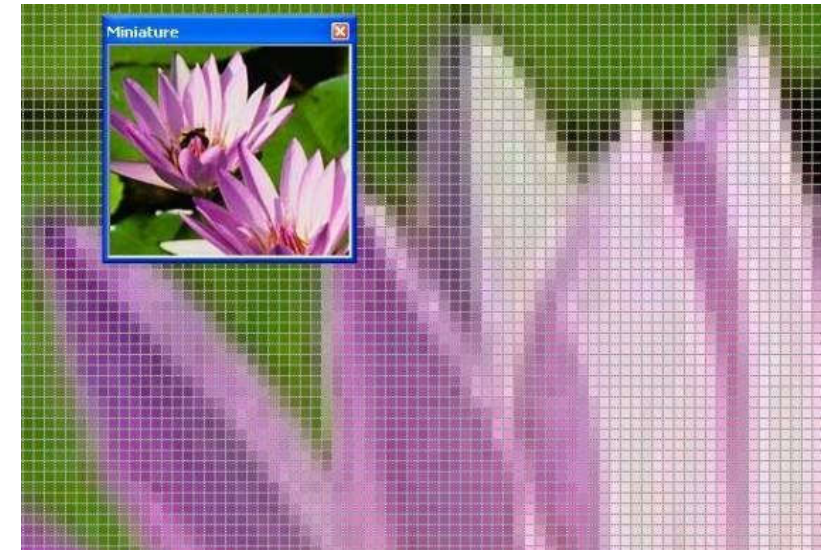
e. Codage d'une image

Le principe du codage d'une image :

- Tout commence par découper l'image en des petits carrés c'est en quelque sorte poser une grille (aussi serrée que possible) sur l'image.

- Deux nombres seront importants pour décrire cette grille : le nombre de petits carrés en largeur et ce même nombre en hauteur.
- Plus ces nombres sont élevés, plus la surface de chaque petit carré est petite et plus le dessin trame sera proche de l'originale.

On obtient donc pour toute l'image un quadrillage comme celui montré ci-dessous pour une partie.



Il ne reste plus qu'à en déduire une longue liste d'entiers :

- Le nombre de carré sur la largeur.
- Le nombre de carré sur la hauteur.
- Suite de nombres pour coder l'information (Couleur) contenue dans chaque petit carré qu'on appelle pixel (**PI**Cture **E**LEMENT) :
 - Image en noir et blanc → 1 bit pour chaque pixel.
 - Image avec 256 couleurs → 1 octet (8 bits) pour chaque pixel.
 - Image en couleur vrai (True Color : 16 millions de couleurs) → 3 octets (24 bits) pour chaque pixel.

La manière de coder un dessin en série de nombres s'appelle une représentation **BITMAP**.

L'**infographie** est le domaine de l'informatique concernant la création et la manipulation des images numériques.

La **définition** : détermine le nombre de pixel constituant l'image. Une image possédant 800 pixels en largeur et 600 pixels en hauteur aura une définition notée 800x600 pixels.

La **profondeur** ou la dynamique d'une image est le nombre de bits utilisé pour coder la couleur de chaque pixel.

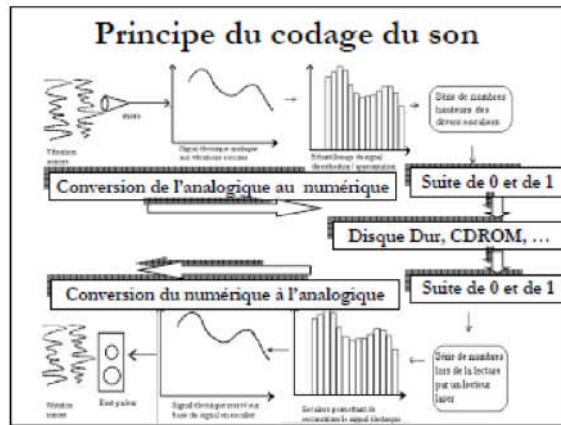
Le **poids** d'une image (exprimé en Ko ou en Mo) : est égal à son nombre de pixels (définition) que multiplie le poids de chacun des pixels (profondeur).

f. Codage du son

Un son est une vibration mécanique se propageant dans l'air ou dans un autre milieu (fluide, solide...). Lors de la numération, il faut transformer le signal analogique (continu) en une suite de nombres qui seront traités par l'ordinateur. C'est le rôle du convertisseur analogique/numérique.

La numérisation d'un son se réalise en deux étapes :

- **L'échantillonnage** : consiste à prélever périodiquement des échantillons d'un signal analogique selon une période que l'on appellera période d'échantillonnage.
- **La quantification** : consiste à affecter une valeur numérique à chaque échantillon prélevé.



Les Codecs, abréviation de codeur/décodeur, sont des logiciels qui permettent de réaliser ce codage/décodage du son dans un ordinateur. Ces logiciels s'appuient sur les éléments matériels disponibles dans la carte son.

Exercices

Série de TD N°5

(Comptage binaire)

Exercice 1 : (Conversion de bases)

Exprimez le nombre décimal 100 dans toutes les bases de 2 à 9 et en base 16.

Exercice 2 : (Représentation des entiers)

Exprimez les nombres décimaux 94, 141, 163 et 197 en base 2, 8 et 16.

Donnez sur 8 bits les représentations « binaire signé », complément à 1 et complément à 2 des nombres décimaux 45, 73, 84, -99, -102 et -118.

Exercice 3 : (Calcul binaire)

- 1/ Effectuez la soustraction $122 - 43$ dans la représentation en complément à 2 en n'utilisant que l'addition.
- 2/ Multipliez les nombres binaires 10111 et 1011 ; vérifiez le résultat en décimal.

Exercice 4 : (Représentation des réels - virgule fixe)

- 1/ Coder en virgule fixe au format Q8.8 les nombres suivants : 23.375, 100, 365.2, -54.65.
- 2/ Décoder le nombre $6C60_{(16)}$ en virgule fixe dans le format Q10.6.
- 3/ Décoder le nombre $D7EA_{(16)}$ en virgule fixe dans le format Q8.8.
- 4/ Déterminer le nombre maximum représentable en virgule fixe sur le format Q4.4.
- 5/ Déterminer le nombre minimal strictement positif représentable sur ce même format.
- 6/ Déterminer le pas de ce format.

Exercice 5 : (Virgule flottante - Norme IEEE754)

Soit une machine où les nombres réels sont représentés sous la forme $(\pm 1, M \cdot 2^{Eb})$ sur 32 bits, numérotés de droite à gauche de 0 à 31 avec :

- Une pseudo-mantisse M normalisée sur 23 bits (les bits 0 à 22).
- Un exposant biaisé, représentant une puissance de 2, codé sur 8 bits (les bits 23 à 31).
- Un bit pour le signe de la mantisse (le bit 31).

- 1/ Déterminer le nombre maximum représentable dans ce format.
- 2/ Déterminer le nombre minimal strictement positif représentable dans ce même format.
- 3/ Donner en octal, la représentation IEEE754 correspondant au nombre décimal -32,625.
- 4/ Mettre sous le format $IIIIII754$, les deux nombres $AE8_{(16)}$ et $9D0_{(16)}$ puis calculer leur valeur décimale.